

Machine Learning Technologies

Generalisation

Validation

Donny Hurley

ATU Galway

Generalisation

Generalisation is the goal of ML

- Want good performance for **new** data
- How good the generalisation is, is how well it performs on previously “unseen data” i.e. data not used to train the set.

Definition

Generalisation is the model's ability to give sensible outputs to sets of input that it has never seen before.

- Root-mean-squared (RMS) error could be used to measure: $E_{RMS} = \sqrt{2E(\mathbf{w}^*)/N}$

Back to previous idea of checking how far away pred is away from the original y, what is the problem?

We are checking the *Generalisation* of the model by using data it was trained on, i.e. it is NOT previously unseen.

Our model may be **overfit** to the training data.

- So you chose a model, and did your training and reduced the training loss to 0. Great right???
- No!
- This is the same as a student learning off the answers at that back of the book.
- They may still do well in the exam but if they are given different questions, they're in trouble.
- They have not generalised their learning, they have memorised specific examples.
- In Machine Learning this is called Over-fitting to the data.

- The opposite problem is under-fitting.
- You can't even seem to reduce your training loss to anything reasonable.
- This is like the student not being smart enough to do the exercises.
- In machine learning this is generally a symptom of not having a complex enough model (Student not smart enough).
- Students not being smart enough is unlikely in the real world, as each of us is given a complex model to work with (The Brain).
- Other causes of under-fitting in machine learning are poor training mechanisms or an inappropriate loss function.
- This is more like the student who sits in the library all day long - looking at social media and can't figure out why they still can't do the examples.

Definition

Underfitting occurs when a statistical model or machine learning algorithm cannot capture the underlying trend of the data.

Intuitively, underfitting occurs when the model or the algorithm does not fit the data well enough. Underfitting is often a result of an excessively simple model.

Definition

*Overfitting occurs when a statistical model or machine learning algorithm captures the **noise** of the data. Intuitively, overfitting occurs when the model or the algorithm fits the training data too well.*

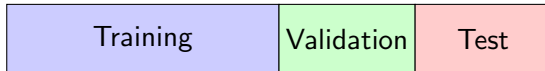
Overfitting is often a result of an excessively complicated model.

Training, Validation, and Test Sets

- **Training set:** used to fit model parameters.
- **Validation set:** used during model development for hyperparameter tuning and model selection.
- **Test set:** used *once* at the end to estimate generalisation performance.

Using the test set during model tuning or selection leads to optimistic and misleading performance estimates.

Dataset Splits (Diagram)



It does not matter what type of models we are building the process is similar.

- Have a set of data.
- Pick base form of models to use.
- **Train** the data on those models
 - If you are trying to choose precisely which model to use - this is where to do it
 - Even if you are comparing different model types (Regression vs NN vs Random Forest etc.) this is the stage to make the decisions. (Using a validation set!)
- **Test** the final chosen model.

Note: no matter how good the model, almost always the more datapoints the better. (The more tuples of x, y the better, having too many independent variables/features can cause issues.)

Building a model

We have a set of data, that is a pair of information. We know the correct answer for the set.

- X is the set of all inputs (a matrix).
- y is the set of all corresponding response variables.
- A particular $y^{(i)}$ is the result for a particular $\mathbf{x}^{(i)}$.

```
1 from sklearn.linear_model import LinearRegression
2
3 lr = LinearRegression()
4 lr.fit(X, y)
5 pred = lr.predict(X)
```

Testing the performance of the model

```
1 lr.fit(X, y)
2 pred = lr.predict(X)
```

In the code, we have created a model called *lr*.

- *lr* was **trained** on the set of *known* information.
- Recall from regression, the predicted values are not exactly the same as what is inputted
- So *pred* is not the exact same as *y* but they hopefully are close.
- So we could test performance by checking how far away *pred* is away from the original *y*
- Does anyone see a problem with this?

So how do we actually test our model

We use a **test** set of data, that we know the answers.

- It does not matter if the response variable is a number (regression) or a quality (category/classifier), it's a similar idea.
 - We may use different formulae to determine accuracy and performance.

Our test set is kept separate from the training set.

When it is used to check performance of the model, it is brand new information to the model. (while we still know the correct answers)

```
1 lr.fit(X_train, y_train)
2 pred = lr.predict(X_test)
3 #Compare pred with y_test
```

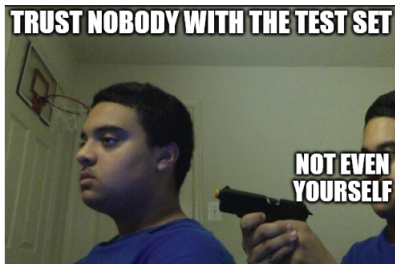
Compare pred with the known results for the test set and it will give allow us to check performance.

Sometimes when trying to solve a problem using you are given an explicit *Training Set* and a separate *Test Set*.

The Training set is used to fit the model (learn the parameters).

The Test set is used to Evaluate the model (check its performance/generalisation).

This is fine and you use them that way. However, often you are just given one set of data and it's your job to decide the best method to fit and train with the large set of data.



This is very very important

The Test Set is Only For the Final Evaluation at the end after you have made all your decisions

If the Test Set is used for any decision making, it is now contaminated

Even if it seems your lecturer is using the test set in lectures/lab books, it is still wrong to use it to make any choices in model selection.

Some Important Notes About the Test Set

- You need to guard the test set carefully, from yourself, and use it only sparingly.
- The Test set should be kept apart and **almost never** used.
- It must not be used in trying to choose any hyper-parameter (polynomial, I'm coming to this) for a model
 - If it was used in selection, it would no longer be unseen data.
- It should never be used when trying decide between say a SVM or a Random Forest model
 - As again, it would no longer be unseen data when making a choice.
- Use it only for overall evaluation.
- ML testing competitions keep the test set hidden and private.

Let's say we start with a big set of data that we have collected. This is a sample of size m .

- For each item i in the data we know the $\mathbf{x}^{(i)}$ vector (features/independent variables)
 - We also know the equivalent $y^{(i)}$ response.
 - We split the data into two sets, a training set and a test set
 - Almost always the training set is going to be significantly larger than the test set.
 - 80/20, 75/25 are often good splits.
 - Randomly select which items are in the training set and which are in the test set.
- Very Important.**
- **Fit** the model on the **training** set.
 - **Evaluate** the model on the **test** set.

Train/test Split

```
1 from sklearn.model_selection import train_test_split
```

is a method built into sklearn to split a dataset into training and test sets randomly.

```
1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size  
    =0.25)
```

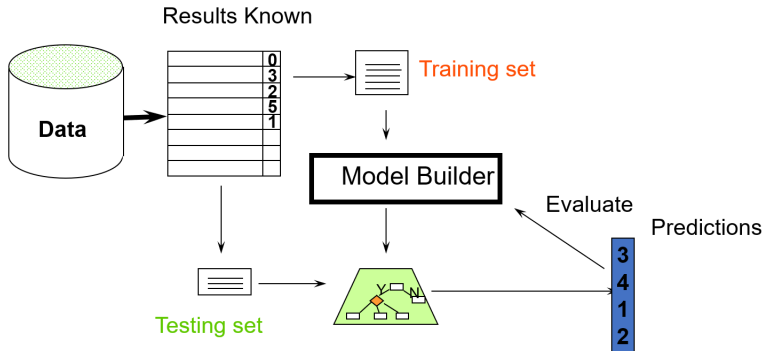
will make a test size of 25% of the data (which is the default anyway)

```
1 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size  
    =0.8)
```

can specify it that way too if you prefer.

The return values will be numpy arrays.

Full model procedure



Deciding the complexity of the model

- When dealing with relatively simple models, (that take only minutes-hours to train), then you can afford to try many different model sizes.
- But you can't just pick the one that gets the lowest training error, we've already shown that can lead us wrong.
- Instead we need a mechanism to check our loss on data that the training algorithm doesn't see.
- The technique we will be looking at is called Hyper-parameter tuning.

Definition

Hyper-parameters are parameters that are chosen rather than learned.

Examples of Hyper-parameters are:

- the degree of your model, e.g. linear, quadratic, n-degree polynomial.
- Which features to use in your model? (Is every bit of information needed?)
- The Learning rate α
- Amount of Regularisation
- The number of layers/neurons in a Neural Network

Polynomial Curve Fitting

- What form is $y(x)$?
 - Try polynomials of degree M .

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M$$

- This is the **hypothesis space**.
 - Note $\mathbf{w}$ is a vector (that's why bold). $\mathbf{w} = (w_0, w_1, \dots, w_M)$
- How do we measure success?
 - Sum of squared errors

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [y(x_n, \mathbf{w}) - t_n]^2$$

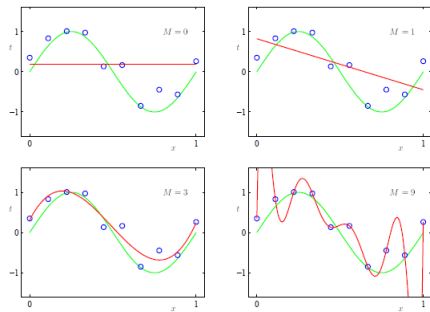
or RMS

- Among functions in the class, choose that which minimises this error.

The degree of polynomial is a *hyperparameter*.

We really want the degree that captures the trend of the data best.

Which degree of Polynomial?



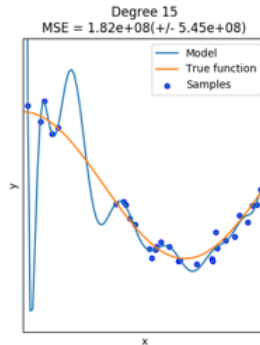
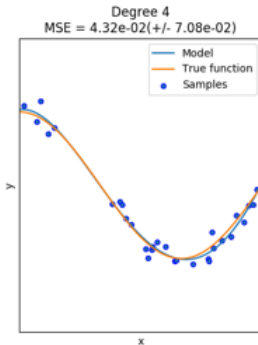
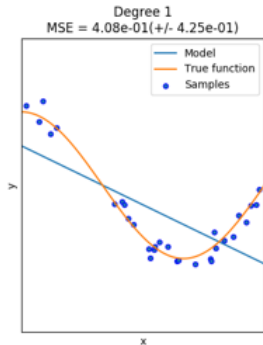
The green line is the "ideal" model, the red line is the shape of the model built.

- If $M = 9$ then $E(\mathbf{w}^*) = 0$: This is **over-fitting**

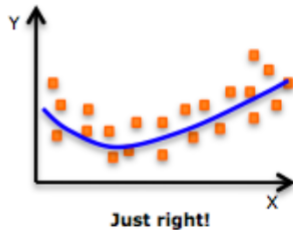
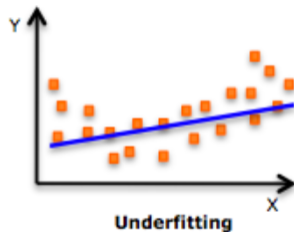
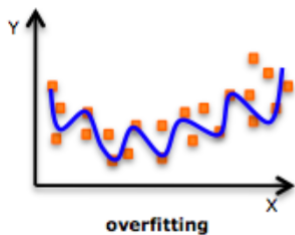
Polynomial regression – overfitting vs underfitting

One of the biggest problems is either overfitting the model or underfitting the model

- Overfitting means the model is too sensitive to the variables you used to "train" the model
- Underfitting means the opposite, too few terms.



Better picture

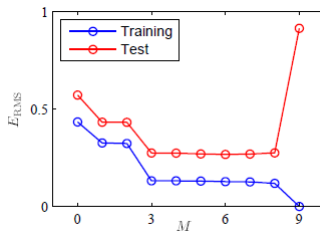


Be aware of these possibilities when creating a regression model

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M$$

Here is the root-mean-squared for each M

- 1 Seeing the performance of the training set vs the model (known information)
- 2 Seeing the performance of the test set vs the model (previously unseen)



The point is, the model performs very very well at $M = 9$ for the training set. It performs very very poorly at $M = 9$ for the test set. i.e. it is overfit at $M = 9$.

Selecting the HyperParameter

So we need to have some method of selecting the hyperparameters.

Note: There can be many different methods for selecting hyperparameters that can be quite complicated. In this level 8 course, I will only demonstrate one real method. I will take shortcuts a little. In the future, if you go into this area, you will have more learning to do.

Keeping the Test Data separate:

- Split training data into **training set** and **validation set** (70/30 split is often used)
- Train different models (e.g. diff. order polynomials) on training set
- Iterate through all possible hyper-parameters (e.g. polynomial $M=1$, then $M=2$, then $M=3$)
 - 1 Fix the hyperparameter ($M=1$ to start)
 - 2 Train a model using the training set
 - 3 Validate the model using the validation set
 - 4 Store the result for that hyperparameter
 - 5 Repeat from step 1 with the next hyperparameter
- Choose model (e.g. order of polynomial) with minimum error on validation set
- (not always done but often) Retrain the model on the complete training data (training set + validation set).

Now we need to be a little careful here.

- Hyperparameters are parameters of the model that are not trained, i.e. we choose them.
- As we can see though with hyper-parameter tuning, we are not exactly training them but we are not choosing them either.
- The tuning that takes place is in some way using the validation set data.
- It may not see the data directly but it is getting a signal from it.
- In effect the validation data is compromised.
- It doesn't give us a true picture of the generalisation of the algorithm.
- For this we need a test set so that we can calculate a test error, that is only used after we have chosen our final model.

Getting a Validation Set

If we have a lot of data, we can go with the above and just use a static validation set to choose our model. `train_test_split` can be used a second time here's an example:

```
1 X_train , X_test , y_train , y_test = train_test_split(X, y, test_size
    =0.2)
2
3 X_train , X_val , y_train , y_val = train_test_split(X_train , y_train ,
    test_size=0.1)
4
5 # Or could do X_train_sm , X_val , y_train_sm , y_val = train_test_split(
    X_train , y_train , test_size=0.1)
```

- At each step train the model using `X_train`
- Validate each hyperparameter using `X_val`
- Make the choice based off that
- Retrain the model using both `X_train` and `X_val` together
- Evaluate the model using `X_test`

Validation vs Testing

You choose your model using validation (even comparing different algorithm types, use validation).

You can evaluate your final model with the test set.

Using a static Validation Set can introduce Bias. What if the validation set is not representative? If we have limited data this is more likely to occur.

The idea of **Cross-Validation** is to do multiple runs at validation

- Cross-validation creates S groups of data, use $S - 1$ to train, other to validate
- Do a number of runs with these different groups (depending on Cross-Validation method chosen)
- Average over all the scores for the different training/validation splits
- Choose the hyperparameter that has the lowest average over all runs

There are many types of cross-validation as time is limited, here's one

k-fold Cross Validation

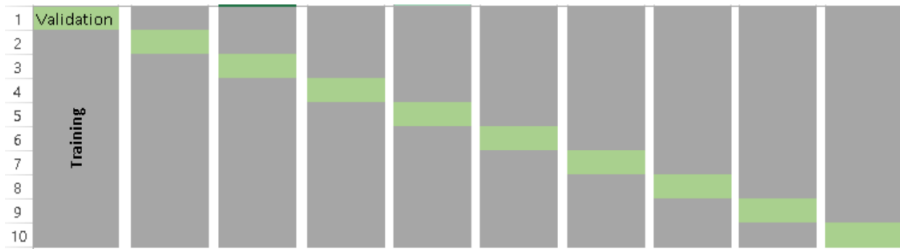
- Randomly split your entire dataset into k "folds"
- For each k -fold in your dataset, build your model on $k - 1$ folds of the dataset. Then, evaluate the model to check the effectiveness for k th fold
- Record the error you see on each of the predictions
- Repeat this until each of the k -folds has served as the validation set
- The average of your k recorded errors is called the cross-validation error and will serve as your performance metric for the model

(5 and 10 are usually often good choices for the number of folds - the default in sklearn is 5)

Note

This is not feasible for more complicated models like Deep Learning, for those you have to just use a static validation set.

10-fold Cross Validation



Final procedure yet?

Keeping the Test Data separate:

- Iterate through all possible hyper-parameters (e.g. polynomial $M=1$, then $M=2$, then $M=3$)
 - ① Fix the hyperparameter ($M=1$ to start)
 - ② Randomly split the training data in k -folds
 - ① Train a model using $k - 1$ folds of the dataset.
 - ② Evaluate the model using the remaining fold.
 - ③ Record the error.
 - ④ Repeat until all folds have been a validation set.
 - ③ Take the average error of all folds and store that result for the particular hyperparameter
 - ④ Repeat from step 1 with the next hyperparameter
- Choose model (e.g. order of polynomial) with minimum (average) error on the folds
- (not always done but often) Retrain the model on the complete training data (training set + validation set).

You choose your model using validation, you can evaluate your final model with the test set.

A procedure

- ① Split your labelled/known results into **training set** and **test set**
- ② Select your base model (linear regression, nearest neighbours etc.)
- ③ Tune the hyper-parameters of the model using Cross Validation
- ④ Train the model (with chosen parameters) with the training set
- ⑤ Compare the different types of models using some form of validation - make final decision.
- ⑥ Evaluate the final model using the test set

Multiple Hyperparameters

- It is clear to see that if you have multiple hyper-parameters, it is difficult to choose the optimum for each as you have to hold all other hyper-parameters steady while you test one of them.
- What value do you hold them steady at?
- Well you have to try with every combination. (GridSearchCV can help with this and we'll see it later)
- When dealing with large data sets that take weeks to train you can see that this becomes in-feasible.
- With small data sets it can be difficult to split into train-val-test and still have enough training data.
- There is no perfect solution.

Whenever we discuss model prediction, it's important to understand prediction errors. There is a tradeoff between a model's ability to minimise bias and variance.

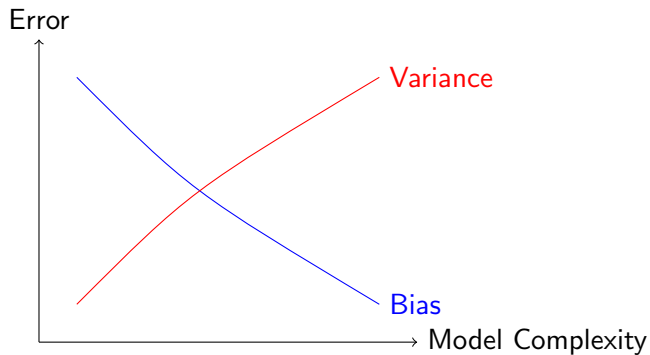
Definition

Bias is the difference between the average prediction of our model and the correct value which we are trying to predict. Model with high bias pays very little attention to the training data and oversimplifies the model. It always leads to high error on training and test data.

Definition

Variance is the variability of model prediction for a given data point or a value which tells us spread of our data. Model with high variance pays a lot of attention to training data and does not generalise on the data which it hasn't seen before. As a result, such models perform very well on training data but have high error rates on test data.

Bias–Variance Trade-off (Diagram)



Overfit/underfit

Overfitting is the case where the overall cost is really small, but the generalisation of the model is unreliable. This is due to the model learning "too much" from the training data set. (low bias, high variance)

Underfitting is the case where the model has "not learned enough" from the training data, resulting in low generalisation and unreliable predictions. (high bias, low variance)

Bias-variance trade-off. What is the right measure?

- Depending on the model at hand, a performance that lies between overfitting and underfitting is more desirable.
- This trade-off is the most integral aspect of Machine Learning model training.
- Machine Learning models fulfil their purpose when they generalise well.
- Generalisation is bound by the two undesirable outcomes — high bias and high variance.
- Detecting whether the model suffers from either one is the sole responsibility of the model developer.

- One of the major problems with over-fitting is that it does not just mis-classify new data but it can often do so with very high confidence.
- When the model is of more modest complexity it may have certain samples in the training set that end up on the wrong side of the decision boundary (classification) or are a long way from the model (regression).
- However the model can report this.
- The model can say, I predict this but I am only $x\%$ sure (classification) or there is x -tolerance with this prediction (regression).

Seeing Overfitting/underfitting

Often people compare the score for the training set vs the score for the validation set to see if either of these are happening

```
1 lr.fit(X_train, y_train)
2
3 train_score = lr.score(X_train, y_train)
4 val_score = lr.score(X_val, y_val)
```

- If the training score is very poor, then the model is *underfit*
- Almost always the training score will be better than the validation score.
- If there is a large difference between training score and validation score (training score much better), then the model is *overfit*

Dealing with over/underfitting

It is a constant issue but there are some common techniques

For under-fitting you either need to train for longer or use a more complex model. It may also require you to look at your training mechanisms and loss function. Are they appropriate to the problem?

For over-fitting you could:

- Make the model less complex.
- Decrease the number of features. Particularly if there is a large number of features compared to the number of training samples. But you are also throwing away information with this method. Choose wisely what to throw out.
- Get more data.
- Often use regularisation (later we'll talk about this).

Another term that applies in general to “most” Machine Learning systems is the idea of Regularisation.

Definition

Regularisation is a technique used for tuning the function by adding an additional penalty term in the error function. The additional term controls the excessively fluctuating function such that the coefficients don't take extreme values.

This technique of keeping a check or reducing the value of error coefficients are called shrinkage methods or weight decay in case of neural networks.

Some Points:

- Regularisation is a more general way of avoiding overfitting.
- It works by adding a term to the loss function which adds loss if the model is not of a preferred type.
- In particular we don't want the weights to become very large.
- A large weight can change by a large amount very quickly.
- As differences in the magnitude of weights can quickly spiral out of control and individual weights influence can quickly exceed their importance.
- For this reason we add a penalty to the loss function that is related to the size of the weights.

$$MSE(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - f_{\mathbf{w}}(\mathbf{x}^{(i)}))^2$$

- It can happen that when “optimising” this Loss function, some of the weights may become very large.
- That means the feature corresponding to these large weights will “increase” in their importance
- This can cause overfitting.

So what can we do with these large weights?

Mean Squared Error - l2 (Ridge Regression)

We add a regularisation term (λ) to our Loss function that penalises large weights

$$L(\mathbf{w}) = MSE(\mathbf{w}) + \lambda \sum_{j=1}^n w_j^2$$

This means that if any particular w_j is large, the overall Loss will be large too.

This is known as the L2 norm. (squared magnitude)

It does not matter if we are using MSE or any other loss function, we add on the regularisation term to the end.

Here is the MSE with regularisation fully written out

$$L(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - f_{\mathbf{w}}(\mathbf{x}^{(i)}))^2 + \lambda \sum_{j=1}^n w_j^2$$

(some places may write this slightly differently), where n is the number of features and m is sample size. Notice we do not include the bias term (w_0)

L2 Regularisation (Regularization)

The **effect** of regularisation is to make it so the model prefers to learn small weights, all other things being equal.

Large weights will only be allowed if they considerably improve the first part of the cost function.

The relative importance of the two elements of the compromise depends on the value of λ :

- when λ is small we prefer to minimize the original cost function
- when λ is large we prefer small weights.

Large λ moves in the direction of underfitting, and small λ moves in the direction of overfitting.

So it is another hyperparameter that we have to choose - perhaps via cross-validation. Additionally Gradient Descent applies just as it does without regularisation, it just has an extra term when calculating the updates/gradients.